Functions and Getting Help

Course: Kaggle Learn · Python Instructor: Colin Morris · Lesson: Functions and Getting Help

1. Introduction to Functions

Python provides many built-in functions, such as `print()` and `abs()`, that perform common tasks. However, built-in functions alone are not enough for every programming problem. Python allows us to define our own functions to organize code, reuse logic, and perform specific tasks.

In this lesson, we will learn how to:

- Use `help()` to understand unfamiliar functions.
- Define and call custom functions using `def`.
- Return values using `return`.
- Document functions using docstrings.
- Work with functions that do not return values.
- Set default argument values.
- Pass functions as arguments to other functions.

2. Getting Help with `help()`

The `help()` function displays documentation about a Python object or function. It is particularly useful when you forget what a function does, what arguments it accepts, or how to use it.

2.1 Using `help()` with Built-in Functions

Suppose you have forgotten how `round()` works. You can inspect its documentation using:

help(round)

Output:

```
Help on built-in function round in module builtins:
round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None.
```

```
    Otherwise the return value has the same type as the number.
    ndigits may be negative.
```

The help output provides two important things:

1. Function signature: `round(number, ndigits=None)` shows the function's name and the arguments it accepts.
2. Description: Explains what the function does and how its arguments behave.

Syntax	Meaning
`number`	The number to round.
`ndigits=None`	An optional argument specifying the number of decimal places. Its default value is `None`.

For example:

```
print(round(3.14159, 2))
print(round(3.14159))
```

Output:

```
3.14
3
```

When `ndigits` are omitted, `round()` rounds to the nearest integer.

2.2 Passing the Function Itself vs. Calling It

A common mistake is passing the result of a function call to `help()` instead of passing the function itself.

Correct:

```
help(round)
```

This displays documentation about the `round` function.

Incorrect for looking up the function's documentation:

```
help(round(-2.01))
```

Python evaluates expressions from the inside out:

1. It evaluates `round(-2.01)`, producing the integer `-2`.
2. It then calls `help(-2)`, displaying help for the resulting integer object rather than for `round()`.

Important rule: Use `help(function_name)` to look up a function's documentation. Do not add parentheses containing arguments unless you specifically want help about the result of that call.

2.3 Using `help()` with Complex Functions

Consider the built-in `print()` function.

```
help(print)
```

Relevant output:

```
Help on built-in function print in module builtins:

print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.

    Optional keyword arguments:
    file:  a file-like object (stream); defaults to the current sys.stdout.
    sep:   string inserted between values, default a space.
    end:   string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.
```

The documentation reveals several optional arguments:

Argument	Purpose	Default value
`sep`	String inserted between printed values.	`' '` (space)
`end`	String appended after the final value.	`'\n'` (newline)
`file`	Destination stream for the output.	`sys.stdout`

`flush`	Controls whether the output stream is forcibly flushed.	`False`

For example:

```
print(1, 2, 3, sep=' < ')
```

Output:

```
1 < 2 < 3
```

Here, `sep=' < '` tells Python to insert `<` between the values instead of the default space.

Without specifying `sep`:

```
print(1, 2, 3)
```

Output:

```
1 2 3
```

The default separator is a single space.

Key takeaway: `help()` is a valuable tool for learning unfamiliar functions, understanding argument names, and discovering optional parameters.

3. Defining Your Own Functions

Built-in functions are useful, but we often need to create functions that perform tasks specific to our programs.

Python uses the `def` keyword to define a function.

3.1 Function Definition Syntax

```
def function_name(parameters):    # Function body
    return result
```

Component	Purpose
`def`	Keyword used to define a function.
`function_name`	Name used to identify and call the function.
`parameters`	Variables that receive input values.
`:`	Marks the beginning of the function body.
Indented block	Contains the statements executed when the function is called.
`return`	Sends a result back to the calling code and exits the function.

3.2 Example: Defining `least_difference()`

Consider a function that calculates the smallest absolute difference between any two of three numbers.

```
def least_difference(a, b, c):
    diff1 = abs(a - b)
    diff2 = abs(b - c)
    diff3 = abs(a - c)
    return min(diff1, diff2, diff3)
```

This function accepts three parameters: `a`, `b`, and `c`.

How it works:

1. `abs(a - b)` calculates the absolute difference between `a` and `b`.
2. `abs(b - c)` calculates the absolute difference between `b` and `c`.
3. `abs(a - c)` calculates the absolute difference between `a` and `c`.
4. `min()` selects the smallest of the three differences.
5. `return` sends that smallest difference back to the caller.

For example, if the inputs are `1`, `10`, and `100`, the differences are `9`, `90`, and `99`. The smallest is `9`.

3.3 Calling a Custom Function

Defining a function does not automatically execute its body. The function must be called.

```
print(
    least_difference(1, 10, 100),
    least_difference(1, 10, 10),
    least_difference(5, 6, 7),
)
```

Output:

```
9 0 1
```

Explanation:

Function call	Absolute differences	Returned value
`least_difference(1, 10, 100)`	`9`, `90`, `99`	`9`
`least_difference(1, 10, 10)`	`9`, `0`, `9`	`0`
`least_difference(5, 6, 7)`	`1`, `1`, `2`	`1`

Python also permits a trailing comma after the final argument in a function call.

least_difference(5, 6, 7,)

This is valid Python syntax.

3.4 Understanding `return`

The `return` keyword has two main effects:

- It immediately terminates the current function execution.
- It passes the specified value back to the code that called the function.

Example:

```
def square(number):
    return number * numberresult = square(5)print(result)
```

Output:

```
25
```

The function calculates `5 * 5`, returns `25`, and stores the returned value in `result`.

Statements placed after an executed `return` are not executed.

```
def example():
    return 10
print("This will not execute")
print(example())
```

Output:

```
10
```

The function exits as soon as it encounters `return 10`.

4. Docstrings: Documenting Functions

A function's code may explain how it works, but it does not always explain its intended purpose. A docstring provides a description of a function for programmers who use or maintain it.

4.1 Writing a Docstring

A docstring is a string placed immediately after the function header. It is commonly written using triple double quotes (`""" """`), and it can span multiple lines.

def least_difference(a, b, c): """Return the smallest difference between any two numbers among a, b and c. >>> least_difference(1, 5, -5) 4 """ diff1 = abs(a - b) diff2 = abs(b - c) diff3 = abs(a - c) return min(diff1, diff2, diff3)

The docstring describes the function's purpose and includes an example of its expected behavior.

4.2 Viewing a Docstring with `help()`

```
help(least_difference)
```

Relevant output:

```
Help on function least_difference in module __main__:

least_difference(a, b, c)
    Return the smallest difference between any two numbers
    among a, b and c.

    >>> least_difference(1, 5, -5)
    4
```

Python displays the docstring as part of the function's help documentation.

Without a docstring, Python can still show the function's signature, but it cannot automatically generate a useful English explanation of the programmer's intended purpose.

4.3 Examples Inside Docstrings

The following lines illustrate an example call and its expected result:

>>> least_difference(1, 5, -5)4

- `>>>` represents a command entered in an interactive Python shell.
- `4` represents the expected output.

Python does not execute this example merely because it appears in the docstring. It is documentation for the reader.

Including examples in docstrings can make functions easier to understand and use.

Best practice: Write meaningful docstrings for functions, especially when the code will be reused, shared, or maintained.

5. Functions That Do Not Return Values

Python allows functions to execute without explicitly returning a value.

Consider the following version of `least_difference()`:

```
def least_difference(a, b, c):
"""Return the smallest difference between any two numbers    among a, b and c.    """
   diff1 = abs(a - b)
   diff2 = abs(b - c)
   diff3 = abs(a - c)
   min(diff1, diff2, diff3)
```

Notice that the function calculates the minimum but does not return it.

Now call the function:

```
print(
    least_difference(1, 10, 100),
    least_difference(1, 10, 10),
    least_difference(5, 6, 7),
)
```

Output:

```
None None None
```

5.1 Understanding None

If a function finishes without executing a `return` statement that supplies a value, Python automatically returns the special value `None`.

`None` represents the absence of a value. It is similar to `null` in some other programming languages, although the languages' precise semantics differ.

The expression `min(diff1, diff2, diff3)` calculates the answer, but its result is discarded because it is neither returned nor otherwise used.

Corrected version:

```
def least_difference(a, b, c):
    diff1 = abs(a - b)
    diff2 = abs(b - c)
    diff3 = abs(a - c)
    return min(diff1, diff2, diff3)
```

5.2 Functions With Side Effects

Not every useful function needs to return a value. Some functions are useful because they cause a side effect—an observable action other than returning a result.

Examples include:

- Displaying text using `print()`.
- Displaying documentation using `help()`.
- Writing data to a file.
- Modifying data or changing some external state.

For example:

```
mystery = print()
print(mystery)
```

Output:

```
None
```

The first `print()` displays a blank line because no values were provided. It returns `None`, which is assigned to `mystery`. The second `print()` displays that value.

5.3 Returning a Value vs. Performing an Action

Feature	Function that returns a value	Function that performs an action
Main purpose	Produce a result for later use.	Cause a side effect.
Example	`abs(-5)`	`print("Hello")`
Result	Returns `5`.	Displays text and returns `None`.
Can the result be assigned?	Yes, to capture the returned value.	Yes, but the assigned value is `None` if no value is returned.

A function can also perform a side effect and return a value. These categories are not mutually exclusive.

6. Default Arguments

A function can provide default values for parameters, allowing callers to omit those arguments when the defaults are appropriate.

6.1 Default Values in Built-in Functions

Recall the `print()` function:

```
print(1, 2, 3, sep=' < ')
```

Output:

```
1 < 2 < 3
```

If `sep` is not specified, its default value is a single space.

```
print(1, 2, 3)
```

Output:

```
1 2 3
```

The function automatically uses the default separator.

6.2 Defining a Function With a Default Argument

Consider this function:

```
def greet(who="Colin"):
    print("Hello,", who)
```

The parameter `who` has a default value of `"Colin"`.

```
greet()
greet(who="Kaggle")
greet("world")
```

Output:

```
Hello, Colin
Hello, Kaggle
Hello, world
```

Explanation:

Function call	Argument used	Output
`greet()`	Default: `"Colin"`	`Hello, Colin`
`greet(who="Kaggle")`	Keyword argument: `"Kaggle"`	`Hello, Kaggle`

`greet("world")`	Positional argument: `"world"`	`Hello, world`

In the second call, `who="Kaggle"` explicitly names the parameter. In the third call, Python assigns `"world"` to `who` based on its position.

When a function has an unambiguous parameter order, specifying the parameter name is optional.

6.3 Important Rules for Default Arguments

- If an argument is omitted, Python uses its default value.
- If a value is supplied, it overrides the default.
- A parameter with a default value is optional when calling the function.
- In a function definition, parameters without defaults must appear before parameters with defaults.

For example:

```
def describe(name, age=18):
    print(name, age)
describe("Aru")
describe("Aru", 19)
```

Output:

```
Aru 18
Aru 19
```

The first call uses the default age of `18`, while the second supplies `19`.

7. Functions Applied to Functions

Python allows functions to be passed as arguments to other functions. This makes it possible to write flexible and reusable code.

A function can be treated as a value: it can be assigned to a variable, passed to another function, or returned by a function.

7.1 Passing a Function as an Argument

Consider the following functions:

```
def mult_by_five(x):
```

```
    return 5 * x
def call(fn, arg):    """Call fn on arg."""
    return fn(arg)
def squared_call(fn, arg):    """Call fn on the result of calling fn on arg."""
    return fn(fn(arg))
```

Here:

- `mult_by_five()` multiplies a number by five.
- `call()` receives a function `fn` and an argument `arg`, then calls `fn(arg)`.
- `squared_call()` calls the supplied function twice in succession, using the first result as the second call's argument.

Now execute:

```
print(
     call(mult_by_five, 1),
     squared_call(mult_by_five, 1),
     sep='\n',
)
```

Output:

```
5
25
```

Explanation:

First call:

call(mult_by_five, 1)

1. `fn` refers to `mult_by_five`.
2. `arg` is `1`.
3. `fn(arg)` becomes `mult_by_five(1)`.
4. The result is `5`.

Second call:

squared_call(mult_by_five, 1)

1. The inner call evaluates `mult_by_five(1)`, producing `5`.
2. The outer call evaluates `mult_by_five(5)`, producing `25`.

The name `squared_call` does not mean it squares the number directly. It applies the supplied function twice.

7.2 The Newline Character `\n`

In the previous example:

sep='\n'

The `sep` parameter inserts a newline character between the values printed by `print()`.

Syntax	Meaning
`'\n'`	Newline character
`' '`	Single space
`'\t'`	Tab character

For example:

```
print("First", "Second", sep='\n')
```

Output:

```
First
Second
```

7.3 Higher-Order Functions

Functions that operate on other functions—by accepting them as arguments or returning them—are called higher-order functions.

Examples from this lesson include:

- `call()`, which accepts a function and calls it.
- `squared_call()`, which accepts a function and calls it twice.
- The built-in `max()` function when used with its `key` argument.

Higher-order functions make it possible to customize how a general operation behaves without rewriting that operation for every situation.

8. Using `max()` With a Function as `key`

By default, `max()` returns the largest value among its arguments. However, its optional `key` parameter lets us specify a function that determines how the values should be compared.

Instead of comparing the original values directly, Python compares the results of applying the key function to each value and returns the original argument with the greatest key result.

8.1 Example: Finding the Largest Number

```
print(max(100, 51, 14))
```

Output:

```
100
```

The largest number is `100`.

8.2 Example: Finding the Number With the Largest Remainder

Define a function that calculates the remainder after division by five.

```
def mod_5(x):    """Return the remainder of x after dividing by 5."""
    return x % 5
```

The `%` operator calculates the remainder of a division.

```
print(
   'Which number is biggest?',
   max(100, 51, 14),
   'Which number is the biggest modulo 5?',
   max(100, 51, 14, key=mod_5),
   sep='\n',
)
```

Output:

```
Which number is biggest?
100
Which number is the biggest modulo 5?
14
```

8.3 Understanding the `key` Argument

Consider the remainder of each number:

Number	Calculation	Remainder
100	100 % 5	0
51	51 % 5	1
14	14 % 5	4

When we use:

```
max(100, 51, 14, key=mod_5)
```

Python evaluates `mod_5()` for each candidate and compares the resulting remainders.

- `mod_5(100)` returns `0`.
- `mod_5(51)` returns `1`.
- `mod_5(14)` returns `4`.

Since `4` is the largest remainder, `max()` returns the original number `14`.

Important distinction: `max(..., key=mod_5)` returns the original input with the largest key result, not the key result itself.

For example:

```
result = max(100, 51, 14, key=mod_5)
print(result)
print(mod_5(result))
```

Output:

```
14
4
```

The first line is the original number. The second line is its remainder.

9. Built-in Functions, Methods, and Syntax Reference

Function / Syntax	Purpose	Example	Result
`help(function)`	Displays documentation for a function.	`help(round)`	Function signature and documentation
`abs(x)`	Returns the absolute value of a number.	`abs(-5)`	`5`
`round(x, n)`	Rounds a number to a specified precision.	`round(3.14159, 2)`	`3.14`
`min(a, b, ...)`	Returns the smallest value.	`min(4, 2, 8)`	`2`
`max(a, b, ...)`	Returns the largest value.	`max(4, 2, 8)`	`8`
`print(...)`	Displays values.	`print(1, 2, sep='-')`	`1-2`
`def name(...):`	Defines a function.	`def greet():`	Defines `greet`
`return value`	Returns a value and exits the function.	`return x * 2`	Returns the calculated value

`"""..."""`	Documents a function when used as its docstring.	`"""Calculate the sum."""`	Documentation text
`sep=' '`	Sets the separator between printed values.	`print(1, 2, sep='-')`	`1-2`
`end='\n'`	Sets the ending appended by `print()`.	`print("Hi", end='!')`	`Hi!`
`key=function`	Specifies a function used to evaluate candidates in operations such as `max()` and `min()`.	`max(100, 51, 14, key=mod_5)`	`14`
`x % y`	Returns the remainder of division.	`14 % 5`	`4`
`'\n'`	Represents a newline character.	`print("A", "B", sep='\n')`	`A` and `B` on separate lines

10. Important Differences and Comparisons

10.1 Function Definition vs. Function Call

Feature	Function definition	Function call
Purpose	Creates a function.	Executes a function.
Syntax	`def greet():`	`greet()`

When it runs	The function body is not executed merely by defining it.	The function body executes when called.

10.2 `return` vs. `print()`

Feature	`return`	`print()`
Purpose	Sends a value back to the caller.	Displays output on the screen by default.
Makes a value available to the caller?	Yes, if a value is returned.	Not through its return value; `print()` returns `None`.
Example	`return 5`	`print(5)`

10.3 Positional vs. Keyword Arguments

Feature	Positional argument	Keyword argument
How the value is assigned	Based on argument position.	Based on parameter name.
Example	`greet("world")`	`greet(who="world")`
Benefit	Concise when parameter order is clear.	Explicit and often easier to read.

10.4 Regular vs. Higher-Order Functions

Feature	Regular function example	Higher-order function example
Example	`mult_by_five(3)`	`call(mult_by_five, 3)`
Input	Receives a number.	Receives a function and other arguments.
Purpose	Performs a specific operation.	Applies or combines behavior supplied through functions.

10.5 `max()` vs. `max(..., key=...)`

Feature	`max(values)`	`max(values, key=function)`
Comparison	Compares the values directly.	Compares the function's results for each value.
Return value	Largest original value.	Original value with the largest key result.
Example	`max(100, 51, 14)`	`max(100, 51, 14, key=mod_5)`
Output	`100`	`14`

11. Practical Examples

11.1 Create a Function With a Default Argument and a Return Value

```
def calculate_discount(price, discount=10):    """Return the price after applying a percentage discount."""
    final_price = price - (price * discount / 100)
    return final_price
print(calculate_discount(1000))
print(calculate_discount(1000, 20))
```

Output:

```
900.0
800.0
```

Explanation:

- The default discount is `10%`.
- The first call uses the default, so the final price is `900`.
- The second call supplies `20`, so the final price is `800`.
- `return` makes the calculated price available to the caller.

11.2 Use a Function as a Key

```
def last_digit(number):    """Return the last digit of a non-negative integer."""
     return number % 10
numbers = [123, 456, 789, 234]
result = max(numbers, key=last_digit)
print(result)
print(last_digit(result))
```

Output:

```
789
9
```

Explanation:

1. `last_digit()` extracts the last digit using `% 10`.
2. `max()` evaluates the key function for each list element.
3. The last digits are `3`, `6`, `9`, and `4`.
4. The largest last digit is `9`, so `max()` returns the original number `789`.

11.3 Use a Docstring and `help()` Together

```
def add_numbers(a, b):    """Return the sum of two numbers."""
    return a + b
help(add_numbers)
print(add_numbers(4, 6))
```

Relevant help output:

```
Help on function add_numbers in module __main__:

add_numbers(a, b)
    Return the sum of two numbers.
```

Function output:

```
10
```

This example demonstrates how documentation makes a custom function easier to understand and reuse.

12. Quick Reference

Concept	Remember
Getting help	Use `help(function_name)`.
Defining functions	Use `def function_name(parameters):`.
Function body	Indent statements beneath the function header.
Returning values	Use `return value` to send a result back.
Missing return value	A function that finishes without returning a value returns `None`.
Docstrings	Place a descriptive triple-quoted string immediately after the function header.
Default arguments	Set defaults in the definition, such as `def greet(who="Colin"):`.
Keyword arguments	Supply values by parameter name, such as `greet(who="Kaggle")`.
Passing functions	Pass the function name without calling it, such as `call(mult_by_five, 1)`.

Higher-order functions	Functions that accept or return other functions.
Custom comparison	Use `key=function` with functions such as `max()` and `min()`.
Remainder	Use `%`, as in `14 % 5`, which produces `4`.
Newline	Use `'\n'` to insert a line break.
Default print separator	`sep=' '` inserts a space between values.

13. Common Mistakes to Remember

1. Mistake: Writing `help(round(-2.01))` when you want documentation for `round()`.
    - Correction: Use `help(round)`. The former evaluates the function call first and requests help for its result.
2. Mistake: Defining a function and expecting its body to execute immediately.
    - Correction: Call it by writing its name followed by parentheses, such as `greet()`.
3. Mistake: Calculating a result but forgetting `return`.
    - Correction: Use `return result` when the caller needs the calculated value. Otherwise, the function returns `None` if it finishes without returning a value.
4. Mistake: Assuming `print()` returns the text it displays.
    - Correction: `print()` displays text and returns `None`. Use `return` in your own function when you need to provide a result.
5. Mistake: Writing a docstring outside the function body or after executable statements.
    - Correction: Place the docstring immediately after the function header, inside the indented body.
6. Mistake: Assuming example calls inside a docstring execute automatically.
    - Correction: Docstring examples are documentation unless a separate tool or program explicitly executes them.
7. Mistake: Forgetting that a default argument is used when its parameter is omitted.
    - Correction: Define the default in the function header, such as `def greet(who="Colin"):`.
8. Mistake: Passing `mult_by_five(1)` instead of `mult_by_five` when a function expects another function.
    - Correction: Pass `mult_by_five` to supply the function itself; add parentheses only when you want to call it.

9. Mistake: Assuming `max(values, key=function)` returns the largest key result.
    - Correction: It returns the original value whose key result is largest. Call the key function separately if you need that result.
10. Mistake: Confusing `sep` and `end` in `print()`.
    - Correction: `sep` goes between values, while `end` is appended after the final value.
11. Mistake: Assuming a function must return a value to be useful.
    - Correction: Some functions are useful because of their side effects, such as displaying output or writing to a file.
12. Mistake: Forgetting that Python uses indentation to determine the function body.
    - Correction: Indent the statements belonging to the function consistently.

Final takeaway: Functions help you organize reusable code, `help()` and docstrings make that code understandable, `return` makes results available to callers, default arguments simplify function calls, and higher-order functions let you pass behavior around as an argument.